

# CGI Capacity Analyzer for Director Workload and RFP Assignment

**Freya Kan, Qian Yang, Junxian Lin, Yixiao Jing, Jai**  
Master of Data Science in Computational Linguistics  
University of British Columbia

## Abstract

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand director workload and assign new RFPs. We develop a Capacity Analyzer that combines historical CRM opportunity data with proposal and RFP text to support director capacity estimation and RFP assignment. The system is designed as a transparent decision-support tool rather than a replacement for business judgment. Week 1 established the reproducible opportunity data foundation and module skeletons. In Week 2, we extended this foundation into a first integrated prototype by refining the cleaned opportunity dataset, validating scoring fields, implementing an initial director-level capacity scoring workflow, connecting capacity outputs to the Streamlit dashboard, and advancing the RFP pipeline toward sample preprocessing, chunking, and retrieval-style output. In Week 3, we refined the capacity dashboard around a relative-load-first interpretation and shifted the main development focus toward the RFP Assignment Tool, including API environment checks, Azure and Chroma helper scaffolding, fallback retrieval, assignment schema stabilization, heuristic recommendation logic, and Streamlit RFP page integration.

## 1 Introduction

CGI Atlantic / Media Atlantic needs a more consistent, data-informed way to understand director workload and assign new RFPs. Today, capacity may be tracked manually or inconsistently, making it difficult to see which directors are available, near their historical workload norm, overloaded, or well matched to a new opportunity.

This project builds a CGI Capacity Analyzer to support two related business decisions. First, the Director Capacity Dashboard summarizes each director's current workload, historical baseline, relative load, capacity label, and workload drivers. In the Week 3 prototype, `relative_load` is treated

as the primary business-facing metric because it compares a director's current workload against that same director's historical norm. Second, the RFP Assignment Tool is intended to estimate RFP effort, retrieve similar historical proposal examples, and recommend suitable directors based on experience, available capacity, service-solution fit, and risk.

The system is not intended to replace stakeholder judgment. Instead, it provides a transparent and reproducible data-based starting point that CGI stakeholders can review, validate, and adjust. This design choice is important because the project does not currently have manually labeled ground truth for director capacity, RFP effort, or correct director assignment.

## 2 Data

### 2.1 Data Sources

The project uses two anonymized Excel CRM opportunity exports and one JSON file containing historical proposal and response text. The two opportunity exports are referred to as `opps1` and `opps2`. The proposal data contains historical proposal and response document pairs.

All CGI-provided files are treated as sensitive local data. They are not committed to GitHub. Local credentials and API configuration are also excluded from version control.

### 2.2 Opportunity Data

Initial EDA indicates that `opps1` and `opps2` are different views of largely overlapping CRM opportunity data. We use `opps2` as the primary base table because it contains the richer CRM schema and has one unique `opportunity_id` per row in the current export. We use `opps1` as a secondary source: selected fields from `opps1` are collapsed to one row per `opportunity_id` and merged into `opps2`, while `opps1`-exclusive opportunities are appended to the final `opportunity_df`. Fields that appear

| Data source              | Approx. size             | Main contents                                                                                                                                                                     | Role in project                                                                                                                            |
|--------------------------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| opps1 CRM export         | 8,435 rows<br>20 columns | Supplemental product, service-solution, IP, service-solution revenue, opportunity-level fallback revenue, and delivery territory fields; contains repeated opportunity_id values. | Secondary supplemental source. Selected fields are collapsed to one row per opportunity_id before merging into opps2.                      |
| opps2 CRM export         | 8,695 rows<br>33 columns | Richer CRM opportunity schema with owner, status, sales stage, probability, revenue, date, duration, and proposal-related fields.                                                 | Primary base table for opportunity_df. It defines the opportunity-level grain for capacity scoring and dashboard inputs.                   |
| Proposal / response JSON | 19 document pairs        | Historical proposal and response text.                                                                                                                                            | Used for RFP text preprocessing, chunking, local fallback retrieval, and future Azure embedding / Chroma retrieval and assignment support. |

Table 1: Overview of the main project data sources used through Week 3. The files are stored locally and are not committed to GitHub because they contain CGI-provided data.

under different names across the two exports, especially revenue-related fields, require normalization and validation before downstream scoring and are handled through the Week 2 validation and fallback process.

This distinction is important because the downstream capacity model is expected to operate at the opportunity level. During the Week 1 merge implementation, we found that opps1 contains duplicate opportunity\_id values. These duplicates likely reflect service-level or product-level detail rather than distinct opportunities. A direct merge from opps1 into opps2 would therefore multiply some opportunity rows and risk double-counting workload in later capacity scoring.

To preserve the intended data grain, the merge pipeline collapses opps1 supplemental fields to one row per opportunity\_id before joining them to opps2. Duplicate and detail rows from opps1 are preserved in local audit outputs rather than being used to expand the final opportunity\_df.

### 2.3 Proposal and RFP Text Data

The proposal JSON contains 19 historical proposal and response document pairs. These documents are text-heavy and many are too long to embed as single inputs. Therefore, the RFP pipeline prepares proposal text through text extraction, chunking, metadata construction, and retrieval-oriented processing before later Azure embedding and vector-store integration.

By Week 3, the local RFP pipeline can run through a fallback path that extracts text from proposals\_responses.json when available, cre-

ates chunks, applies local deterministic fallback embeddings or in-memory retrieval, and generates a dashboard-facing assignment\_context. The current dashboard fallback path can also use a built-in sample corpus when historical chunks are not supplied. This local path is important because it allows RFP recommendation logic and Streamlit display work to continue while Azure OpenAI deployment details and Chroma persistence behavior are still being confirmed.

The proposal text supports the RFP Assignment Tool by enabling retrieval of similar historical examples. For a new RFP, the intended system retrieves related proposal chunks and combines semantic evidence with structured opportunity features such as director experience, available capacity, service-solution fit, and risk. In the Week 3 implementation, this combination is still heuristic and partly mock-backed when real director capacity data is not passed into the RFP engine. Additional validation focuses on the JSON structure, document length, chunk metadata, and whether proposal chunks can be reliably linked to directors or opportunities.

### 2.4 EDA Findings and Design Implications

The EDA directly influenced several early design decisions. First, because opps2 has the richer CRM schema and preserves the opportunity-level grain more cleanly, it is used as the base table for opportunity\_df. Second, because opps1 contains useful service and product fields but also duplicated opportunity IDs, it is treated as a supplemental source rather than a second base table.

Third, because the proposal documents are long, the RFP pipeline uses chunking before embedding or retrieval-style processing. Finally, because the data does not contain labeled capacity categories or gold-standard RFP assignments, the system uses transparent formula-based scoring and retrieval-supported recommendation logic instead of supervised learning.

In Week 3, this design implication becomes more concrete for the RFP workflow. The team keeps both a preferred Azure OpenAI embedding and Chroma retrieval path and a fallback local retrieval path. The Azure / Chroma path is still pending full end-to-end validation, while the fallback path preserves progress on the RFP Assignment Tool as API credentials, deployment names, API versions, and persisted vector-store behavior are confirmed.

### 3 System Overview

The project has two core deliverables. The first is a Director Capacity Dashboard. This dashboard uses historical CRM opportunity data to estimate each director's current workload, compare the current workload against the director's historical baseline, and display capacity labels such as *Available*, *Near Historical Norm*, *High Load*, *Overextended*, and *No baseline*. In the Week 3 prototype, *relative\_load* is treated as the primary business-facing workload metric because it directly compares a director's current workload against that same director's historical norm.

The dashboard also exposes drivers of the score, including open opportunities, late-stage opportunities, weighted pipeline, and inferred delivery commitments. This is important because stakeholders need not only a final capacity label, but also an explanation of why a director appears available, near their normal workload, highly loaded, or overextended. The Week 3 dashboard update also keeps *capacity\_score* in the backend for compatibility, but the main dashboard interpretation is now based on *relative\_load*.

The second deliverable is an RFP Assignment Tool. For a new RFP, the tool is designed to estimate effort, retrieve similar historical proposal examples, and recommend directors based on experience, available capacity, service-solution fit, and risk. By Week 3, the RFP pipeline has a working local fallback path for text extraction, chunking, local retrieval, and *assignment\_context* generation. Azure OpenAI embeddings, Chroma-backed re-

trieval, real capacity-aware recommendation logic, and full production-quality assignment ranking remain integration targets rather than completed end-to-end functionality.

The current architecture is modular. The opportunity cleaning pipeline produces a local *opportunity\_df* and a cleaned downstream dataset for validation and scoring. The capacity scoring module consumes this data to compute director-level workload and relative-load metrics. The Streamlit dashboard displays capacity metrics and provides an interface for RFP assignment. The RFP pipeline preprocesses proposal text, creates chunks, supports fallback retrieval over historical or sample examples, and produces a dashboard-facing *assignment\_context* schema for downstream recommendation display.

## 4 Weekly Implementation Progress

### 4.1 Week 1: Data Foundation and Module Skeletons

During Week 1, the team focused on building the first reproducible data foundation and creating skeletons for the main system components. The goal was not to finalize the full capacity model, dashboard, or RFP assignment tool. Instead, we prioritized stable data structures, module boundaries, and interfaces that can support later development.

The work was divided across five roles. Yixiao focused on opportunity data merging, schema comparison, and duplicate / unmatched record summaries. Kian focused on validation of revenue, date, duration, probability, status, and sales-stage fields. Lyken focused on the capacity scoring skeleton and director table prototype. Freya focused on the Streamlit dashboard skeleton and RFP page placeholder. Jai focused on the RFP preprocessing and vector store skeleton.

For the opportunity data merge, we implemented a reproducible pipeline that loads the two anonymized CRM Excel exports, normalizes column names, compares schemas, uses *opps2* as the base opportunity-level table, and supplements selected fields from *opps1*. The pipeline uses *opportunity\_id* as the required join key.

A key Week 1 finding was that *opps1* contains duplicate *opportunity\_id* rows. These duplicates likely reflect service-level or product-level detail rather than separate opportunities. A direct merge from *opps1* into *opps2* created one-to-many expansion, which would risk double-counting workload

| Merge statistic               | Value |
|-------------------------------|-------|
| opps1 rows                    | 8,435 |
| opps2 rows                    | 8,695 |
| opps1 unique IDs              | 7,679 |
| opps2 unique IDs              | 8,695 |
| Matched opportunity IDs       | 7,628 |
| opps1-exclusive IDs           | 51    |
| opps2-only IDs                | 1,067 |
| Final opportunity_df rows     | 8,746 |
| Duplicate IDs in final output | 0     |

Table 2: Week 1 opportunity merge audit summary. The final output preserves opportunity-level grain by avoiding one-to-many expansion from duplicated opps1 records.

in downstream capacity scoring.

To preserve the intended opportunity-level grain, we collapsed opps1 supplemental fields to one row per opportunity\_id before merging. Duplicate and detail rows from opps1 are preserved in local audit outputs instead of multiplying the final opportunity\_df. The final local opportunity\_df contains 8,746 rows, matching the expected no-expansion count of 8,695 opps2 base rows plus 51 opps1-exclusive opportunity IDs. The final merged dataset contains zero duplicate opportunity\_id rows.

All generated processed data files are kept local and are not committed to GitHub because they are derived from sensitive CGI data. The reproducible code and tests are committed instead, so other team members with local access to the source files can regenerate the same outputs.

## 4.2 Week 2: From Skeletons to First Integrated Prototype

In Week 2, the team moved from separate data and module skeletons toward a first integrated prototype. The main development path connected the cleaned opportunity data to validation, owner-level capacity scoring, and Streamlit dashboard display. In parallel, the RFP pipeline progressed from placeholder functions toward sample text preprocessing, chunking, and retrieval-style output.

Yixiao refined the Week 1 merge output into a cleaner downstream dataset for validation, scoring, and dashboard development. This work focused on stabilizing cleaned\_opportunity\_df, confirming the required columns for downstream modules, preserving source and quality flags, and preparing an owner-level base summary using opportunity\_owner. The goal was not to re-design the Week 1 merge logic, but to make the

merged data easier for Kian’s validation work, Lyken’s scoring module, and Freya’s dashboard integration to consume.

Kian extended the validation work from basic field inspection to scoring-oriented field assessment. The validation process reviewed revenue fields, date and duration fields, probability, status, sales stage, opportunity\_owner, and opportunity\_manager. The output separates fields that are reliable, fields that are usable with fallback assumptions, and fields that remain risky for formal scoring. This validation layer is important because the capacity model depends on transparent assumptions rather than supervised labels.

Lyken converted the Week 1 capacity\_engine.py skeleton into a first working capacity scoring workflow. The scoring module computes sales load, inferred delivery load, current owner-level load, historical baseline, relative load, bounded capacity score, and capacity labels. The resulting director-level table provides the first version of director\_capacity\_df, including fields such as current\_load, historical\_avg\_load, relative\_load, capacity\_score, and capacity\_label.

Freya updated the Streamlit dashboard from a mock-data skeleton toward an integrated capacity dashboard. The Week 2 dashboard reads either the first director\_capacity\_df output or a schema-aligned sample output with the same expected columns. The dashboard displays capacity score by director, capacity labels, current load versus historical baseline, and an owner-level capacity table. Filters and explanation placeholders were also retained so that the interface can continue to evolve as scoring assumptions are refined.

Jai advanced the RFP pipeline from skeleton files toward an independently runnable prototype. The Week 2 work focused on RFP text preprocessing, chunk generation, chunk metadata structure, and a stable assignment\_context output schema for dashboard integration. The prototype can produce sample chunks and basic or mock retrieval-style output. Full Azure OpenAI and Chroma integration remain later steps, but the Week 2 interface gives the dashboard a stable structure for retrieved examples, recommended directors, risk flags, and explanatory notes.

Overall, Week 2 established the first end-to-end project workflow: cleaned opportunity data supports validation; validation findings inform capacity scoring; capacity scoring produces a dashboard-

| Module           | Owner         | Week 2 update                                                                                                                        | Main output                                                                                                                      |
|------------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Data integration | Yixiao        | Refined the Week 1 merge output into a downstream-ready cleaned dataset.                                                             | cleaned_opportunity_df, column dictionary, quality flags, and owner-level summary.                                               |
| Data validation  | Kian          | Validated fields needed for capacity scoring and dashboard use.                                                                      | Missingness summary, reliable / risky fields, revenue fallback hierarchy, and date / duration assumptions.                       |
| Capacity scoring | Lyken         | Implemented the first working owner-level capacity scoring workflow.                                                                 | director_capacity_df, relative load, capacity score, capacity label, and scoring sanity checks.                                  |
| Dashboard        | Freya         | Connected the dashboard to real or schema-aligned scoring output.                                                                    | Capacity score chart, owner-level table, filters, and explanation placeholders.                                                  |
| RFP pipeline     | Jai           | Built an independently runnable RFP preprocessing and retrieval-style prototype.                                                     | Sample chunks, mock or basic retrieval output, and assignment_context schema.                                                    |
| Consistency pass | Yixiao, Freya | Aligned architecture, capacity scoring, dashboard behavior, RFP retrieval, and output contracts after the latest RFP/capacity merge. | Fixed scoring gates, active-quarter baseline/trend logic, dashboard live-data path, stale RFP retrieval, and related docs/tests. |

Table 3: Week 2 progress from separate skeletons toward the first integrated prototype.

ready director table; and the dashboard begins displaying capacity outputs. The RFP pipeline remains partly independent from the capacity workflow, but it now has a clearer preprocessing and output contract for future recommendation integration.

### 4.3 Week 3: Capacity Dashboard Refinement and RFP Assignment Integration

In Week 3, the team shifted the main development focus from the opportunity data foundation and capacity dashboard prototype toward the RFP Assignment Tool. At the same time, the Director Capacity Dashboard received an important refinement: the dashboard now treats `relative_load` as the primary business-facing workload metric instead of `capacity_score`. This change better communicates overextension because `relative_load` compares a director’s current workload against that same director’s historical norm, while `capacity_score` is capped once a director reaches or exceeds baseline.

The Week 3 capacity dashboard update also clarified the workload formula and label interpretation. Current load is represented as the sum of sales load and delivery load. Sales load uses open opportunities, stage weight, probability, authoritative revenue, and duration weight. Delivery load uses won active deliveries and monthly revenue. Capacity labels are now assigned from `relative_load`: *Available*, *Near Historical Norm*, *High Load*, *Overextended*, and *No baseline*. These thresholds are

project-side prototype choices and should be reviewed by CGI before production use.

The dashboard filtering logic was also refined. Territory and opportunity-owner filters define the selected director population for both current workload and historical baseline. Row-level filters such as status, sales stage, opportunity type, sales model, and created-date range narrow the current workload and trend views but do not recompute the historical baseline denominator. This keeps the baseline stable and makes filtered views answer a clearer business question: how the filtered current workload compares with the selected directors’ normal workload.

In parallel, the RFP pipeline became the main Week 3 development focus. Based on repository review, the local RFP prototype is partially ready: it can process available proposal text, create chunks, run local fallback embedding or in-memory retrieval, and generate a dashboard-facing `assignment_context`. The active demo path is fallback / heuristic rather than Azure-backed. The remaining integration work focuses on confirming Azure API configuration, completing real Azure OpenAI embedding smoke tests, connecting Chroma retrieval end to end, wiring real capacity data into the RFP engine, refining recommendation logic, and continuing Streamlit RFP page display work.

Yixiao coordinated the Week 3 RFP integration and documentation work. This included reviewing which RFP pipeline components cur-

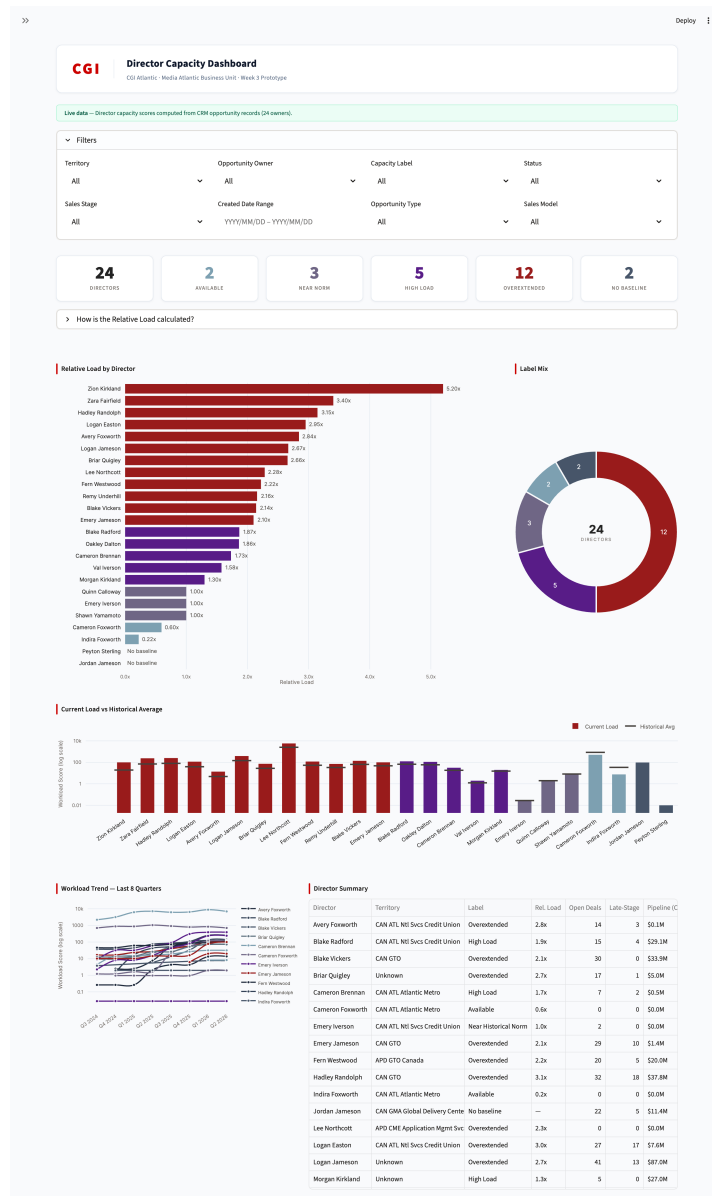


Figure 1: Week 3 Director Capacity Dashboard prototype showing the relative-load-first view, updated capacity labels, label mix, workload comparison, trend view, director-level table, and filters.

rently work locally, confirming the intended API and fallback retrieval behavior, aligning the assignment\_context schema with Jai and Freya, and documenting reproducibility, assumptions, blockers, and risks. A key schema decision is that the backend can keep the current field name effort, while the dashboard displays this object as Estimated Effort. If time allows, estimated\_effort can be added later as an alias without disrupting existing tests.

Kian completed Sprint 3 implementing a reproducible RFP validation workflow over proposals\_responses.json. The validation confirmed 19 top-level records and 46 extracted

documents (19 proposal documents and 27 response documents), with no malformed top-level records in the current local dataset. The work formalized a chunk metadata contract and validated required fields (proposal\_id, chunk\_id, document\_id, source\_type, chunk\_index, text, opportunity\_owner, opportunity\_id) across 1,216 generated chunks, with zero missing required fields. A key finding is that proposal chunks currently cannot be reliably linked to directors or opportunities because opportunity\_owner and opportunity\_id are null in all validated chunks. This limitation is documented explicitly rather than patched with unreliable inferred mapping. De-

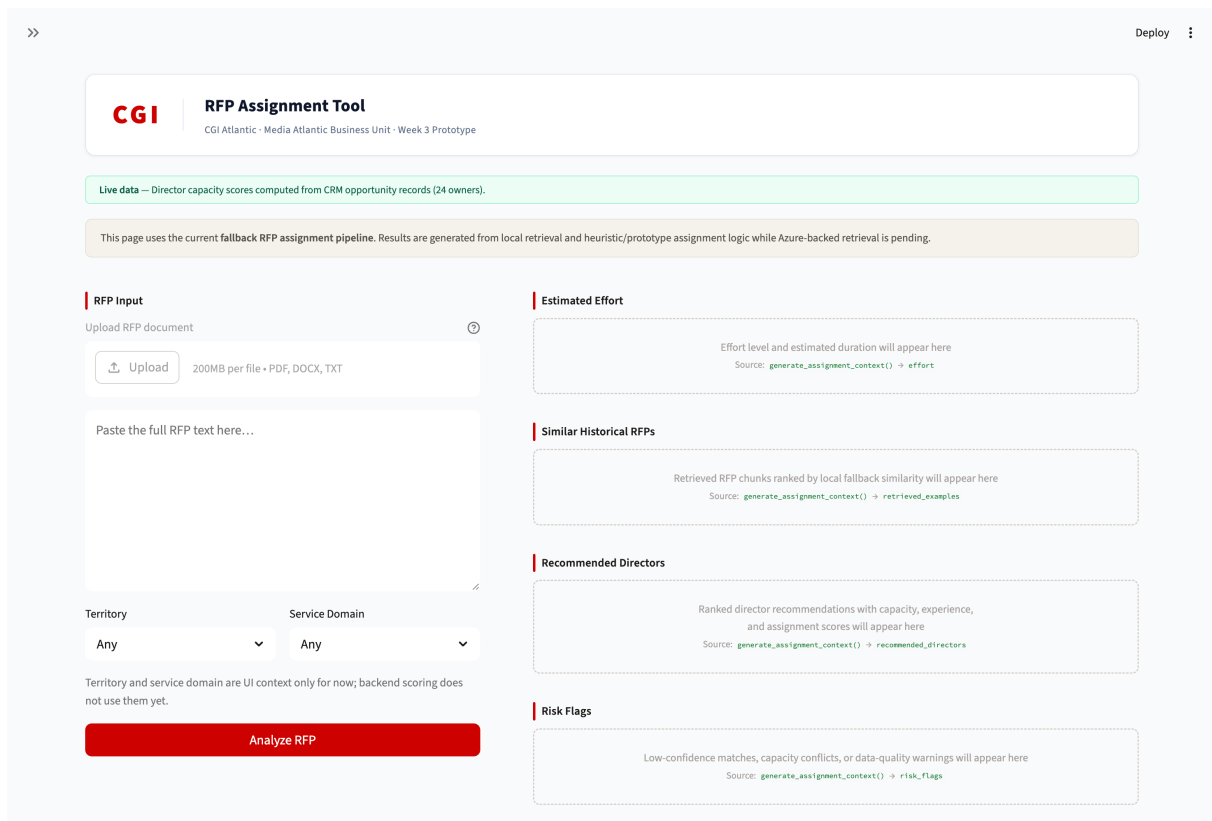


Figure 2: Week 3 RFP Assignment Tool prototype showing the RFP input area and dashboard-facing assignment\_context display for estimated effort, similar examples, recommended directors, and risk flags.

liverables include `src/rfp_data_validator.py`, `scripts/validate_rfp_data_and_export_sample`, `docs/rfp_data_validation.md`, and exported artifacts under `outputs/rfp_validation/` for Lyken/Jai handoff. Export defaults use redacted text and anonymized IDs for safe sharing.

Lyken focused on the Azure embedding and vector-store path. The goal is to move beyond local heuristic embedding toward Azure OpenAI embeddings and Chroma retrieval, while keeping local fallback retrieval available. This work includes confirming environment variables, implementing or updating the Azure client, connecting embeddings to `vector_store.py`, and documenting how the system behaves when Azure is unavailable. Real Azure embedding smoke testing remains blocked until the required API version and embedding deployment name are confirmed. The fallback retrieval path remains available to ensure the RFP pipeline and dashboard workflow can continue even when Azure integration is unavailable.

Freya focused on the RFP page in Streamlit. The Week 3 target is to move the RFP Assignment Tool page beyond a placeholder by displaying assignment\_context fields such as RFP sum-

mary, estimated effort, retrieved examples, recommended directors, capacity signals, match reasons, supporting chunks, and risk flags. The page should also handle empty retrieval results or API failures gracefully and clearly label prototype, heuristic, fallback, or mock outputs.

Jai focused on improving `generate_assignment_context()` and the recommendation logic. The current output uses retrieved examples as supporting evidence and can incorporate capacity fields when director data is supplied, but the Streamlit RFP page still calls the engine without real capacity data. Therefore, current recommendations should be described as heuristic / mock-backed prototype output rather than final capacity-aware assignment guidance.

Overall, Week 3 moved the project from a capacity-dashboard-centered prototype toward a broader integrated decision-support tool. The dashboard now has a clearer relative-load-first business interpretation, while the RFP Assignment Tool has a more explicit path from local fallback retrieval toward Azure embedding, Chroma retrieval, assignment schema stabilization, recommendation logic, and Streamlit display.

| Track                          | Owner  | Week 3 focus                                                                                                                                                                                                      | Main output                                                                                                                              |
|--------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Capacity dashboard refinement  | Freya  | Implemented and documented the <code>relative_load</code> -first dashboard interpretation and updated capacity-label semantics.                                                                                   | Updated dashboard charts, filter semantics, workload formulas, documentation, and tests.                                                 |
| RFP integration / QA           | Yixiao | Coordinated RFP pipeline status, API / fallback behavior, assignment schema, smoke testing, and documentation.                                                                                                    | API usage notes, finalized <code>assignment_context</code> schema, smoke test checklist, integration QA notes, and risks.                |
| RFP data validation            | Kian   | Validated <code>proposals_responses.json</code> , text extraction quality, chunk metadata, and proposal-to-director linkage limitations.                                                                          | JSON structure summary, chunk metadata dictionary, sample chunks, and RFP data validation notes.                                         |
| Azure embedding / vector store | Lyken  | Advanced the code path from local fallback toward Azure embeddings and Chroma while preserving fallback retrieval; real Azure smoke testing remained blocked by missing API version and deployment configuration. | Azure client helper updates, vector store structure, fallback retrieval behavior, and documented blockers for Azure / Chroma completion. |
| RFP dashboard integration      | Freya  | Moved the Streamlit RFP page from placeholder toward displaying <code>assignment_context</code> .                                                                                                                 | RFP input UI, result display, retrieved examples, recommended directors, risk flags, and display notes.                                  |
| RFP assignment logic           | Jai    | Improved <code>generate_assignment_context()</code> toward heuristic recommendation output using retrieval examples, available capacity fields when supplied, and risk flags.                                     | Stable assignment output, heuristic effort logic, recommendation notes, risk flag logic, and test updates.                               |

Table 4: Week 3 progress toward RFP Assignment Tool integration.

## 5 Current Methodology and Prototype Design

The current capacity model is transparent and formula-based. Current workload is represented as the sum of sales load and inferred delivery load. Sales load uses open opportunities, stage weight, probability, authoritative revenue, and project duration. Delivery load uses won active deliveries and monthly revenue as a proxy for delivery commitment. This approach keeps the scoring logic interpretable and avoids using an LLM as the source of truth for numeric capacity scores.

In the Week 3 prototype, `relative_load` is treated as the primary business-facing capacity metric. It is defined as a director’s current workload divided by that same director’s historical average workload. This interpretation is more useful for communicating overextension because it can distinguish a director who is slightly above normal workload from one who is carrying multiple times their historical norm. `capacity_score` remains in the backend output for compatibility, but it is no longer treated as the main dashboard metric because it is capped once `relative_load` reaches or

exceeds the baseline.

Capacity labels are now assigned from `relative_load`. The current prototype labels directors as *Available*, *Near Historical Norm*, *High Load*, *Overextended*, or *No baseline*. These thresholds are project-side calibration choices based on the observed director-level distribution and the business interpretation that carrying at least twice the historical workload should be considered overextended. CGI should review and adjust these thresholds before production use.

The dashboard filtering logic is designed to keep the historical baseline stable. Territory and opportunity-owner filters define the selected director population for both current workload and historical baseline. Row-level filters such as status, sales stage, opportunity type, sales model, and created-date range narrow the current workload and trend views, but do not recompute the historical baseline denominator. This allows filtered views to answer how a selected slice of current workload compares with the selected directors’ normal workload.

The Streamlit dashboard delegates capacity calculations to shared functions in `capacity_engine.py`. For live filtered views, the

interface rebuilds current workload from filtered opportunity rows while keeping the historical baseline anchored to the selected owner / territory population. This keeps the scoring logic centralized in the engine while allowing the dashboard to support interactive filtering.

For RFP assignment, the current prototype combines proposal text preprocessing with a retrieval-oriented output structure. By Week 3, the local fallback pipeline can extract text from `proposals_responses.json` when available, create chunks, run local fallback embedding or in-memory retrieval, and generate a dashboard-facing `assignment_context`. This fallback path is important because it allows recommendation logic and Streamlit display work to continue even when Azure OpenAI or Chroma integration is not fully stable.

The preferred RFP retrieval path remains Azure OpenAI embeddings with Chroma vector-store retrieval. Week 3 development focuses on confirming API and environment usage, implementing or updating the Azure client, connecting embeddings to `vector_store.py`, testing chunk-to-embedding-to-query behavior, and preserving local fallback retrieval. The RFP pipeline therefore has two intended execution paths: an API-backed retrieval path for the final prototype and a fallback retrieval path for robustness during development. At the end of Week 3, the fallback path is the active working path, while the Azure / Chroma path still requires confirmed deployment configuration and end-to-end smoke testing.

The intended RFP recommendation logic combines semantic retrieval evidence with structured signals. Similar historical proposal chunks provide supporting evidence, while director capacity, service-solution fit, risk flags, and available metadata should eventually shape the final recommendation. In the current Week 3 prototype, this logic is still heuristic: the dashboard displays `assignment_context`, including an RFP summary, an effort object, retrieved examples, recommended directors, risk flags, and notes, but real capacity data is not yet wired into the RFP page call. In the dashboard, the backend field effort is displayed as Estimated Effort to avoid user-facing ambiguity.

## 6 Project Management and Next Steps

In Week 3, the team continued the five-role structure established in the first two weeks, but the coordination focus shifted from capacity dashboard integration toward RFP Assignment Tool integration. The main RFP workflow now depends on a clearer handoff sequence: RFP text and chunk metadata support retrieval; retrieval outputs feed `generate_assignment_context()`; the assignment context combines effort, retrieved examples, recommended directors, and risk flags; and the Streamlit RFP page consumes that shared output schema for display.

This coordination structure helped the team move RFP development forward without blocking on a single integration point. Yixiao coordinated API / environment notes, fallback behavior, the `assignment_context` schema, smoke testing, and report documentation. Kian focused on validating `proposals_responses.json`, extracted text, and chunk metadata. Lyken worked on Azure / Chroma helper scaffolding and fallback retrieval behavior while the real Azure embedding path remained blocked by missing deployment configuration. Freya moved the RFP page from placeholder display toward showing assignment outputs. Jai refined `generate_assignment_context()` toward a stable heuristic output that can use retrieval examples, available director fields when supplied, match reasons, and risk flags.

The capacity dashboard also required project management attention in Week 3 because the main business-facing metric changed from `capacity_score` to `relative_load`. This decision affected the dashboard charts, label definitions, tooltips, documentation, and tests. The team documented the workload formulas, label thresholds, filter behavior, stable-baseline interpretation, and remaining review items so that CGI can later confirm whether the temporary business thresholds are appropriate.

The project also required extra consistency work across the Week 2 to Week 3 boundary. At the end of Week 2, Yixiao and Freya completed a consistency pass after the latest RFP and capacity changes were merged, aligning architecture, scoring behavior, dashboard live-data logic, RFP retrieval behavior, output contracts, documentation, and tests. At the start of Week 3, Freya completed the capacity dashboard refinement work around the new `relative_load` interpretation, label thresholds, fil-

ter semantics, and dashboard chart changes.

The next stage of development will focus on making the RFP Assignment Tool more robust and reviewable. In Week 4, the team should confirm the Azure API configuration, deployment names, and API version; complete or document the Azure embedding and Chroma retrieval path; keep the fallback retrieval path available; and continue improving the recommendation logic. The RFP page should be tested with representative sample inputs, including empty retrieval results and API failure cases. The team should also collect screenshots and examples for the report and demo.

For the capacity dashboard, the next steps are to review the `relative_load` thresholds with CGI, confirm whether the label names match business language, and decide whether additional dashboard filters should redefine the historical baseline population in future versions. Because the Director Capacity Dashboard remains the first core deliverable, it should stay stable while RFP functionality is integrated. The RFP Assignment Tool can continue to evolve in parallel as long as its outputs clearly distinguish real, heuristic, fallback, and mock components.

## 7 Current Assumptions and Risks

Several assumptions and risks remain at the end of Week 3. First, `opportunity_id` is still treated as the required join key for opportunity-level merging. Second, `opps2` remains the primary base CRM opportunity table, while `opps1` is used only for selected supplemental fields after being collapsed to one row per `opportunity_id`. Third, `opportunity_owner` is treated as the main director-level entity for MVP capacity scoring, while `opportunity_manager` is retained as optional metadata.

The capacity dashboard still depends on inferred workload rather than measured hours, calendar commitments, or manually confirmed director capacity. Current load is estimated from sales pipeline and inferred delivery signals, so the resulting workload estimates should be treated as directional decision-support indicators rather than exact measurements of effort.

The Week 3 dashboard uses `relative_load` as the primary business-facing workload metric. This improves interpretability, but the current label thresholds remain temporary project-side calibration choices. Labels such as *Available*, *Near Histor-*

*ical Norm*, *High Load*, *Overextended*, and *No baseline* should be reviewed by CGI before production use. In particular, the threshold for *Overextended* reflects the current team interpretation that at least twice the historical workload should be treated as overextension.

The stable-baseline filtering decision also remains an assumption. Territory and opportunity-owner filters define the selected director population for both current workload and baseline, while row-level filters narrow current workload without recomputing the historical baseline. This makes the dashboard easier to interpret, but CGI may later decide that some additional filters should also redefine the baseline population.

The RFP pipeline introduces additional integration risks. Azure OpenAI and Chroma retrieval depend on local environment configuration, API credentials, deployment names, API versions, and service availability. If any of these are unavailable or unstable, the system must fall back to local retrieval so that the RFP page, recommendation logic, and report examples can continue to run. At the end of Week 3, real Azure-backed retrieval should still be treated as pending rather than completed.

The `assignment_context` schema must remain stable across the backend, recommendation logic, and Streamlit display. A known naming risk is that the backend currently uses `effort`, while the dashboard displays this field as Estimated Effort. If the team later adds `estimated_effort` as an alias, the change should preserve existing tests and avoid breaking the dashboard contract.

The RFP Assignment Tool may combine real pre-processing, heuristic logic, fallback retrieval, and mock components during development. These components must be clearly labeled in the dashboard and documentation so that users do not overinterpret prototype recommendations as final production recommendations.

Finally, the project still lacks ground-truth labels for director capacity, RFP effort, and correct director assignment. Evaluation will therefore rely on stakeholder review, threshold sensitivity checks, sanity checks, smoke tests, and selected case studies rather than standard supervised-learning metrics.

## 8 Conclusion

This Week 3 version extends the report from a first integrated prototype update into a more re-

financed decision-support system report. Week 1 established the reproducible opportunity merge, preserved opportunity-level grain, and created the main module skeletons. Week 2 refined the cleaned opportunity data, added validation-oriented field assessment, produced an initial director-level capacity scoring output, connected the dashboard to capacity results, and advanced the RFP pipeline toward preprocessing, chunking, and retrieval-style output.

In Week 3, the team refined the Director Capacity Dashboard by adopting `relative_load` as the primary business-facing workload metric, updating capacity label interpretation, clarifying stable-baseline filter behavior, and documenting remaining threshold review items. At the same time, the project focus shifted toward the RFP Assignment Tool, including API and environment checks, Azure / Chroma helper scaffolding, fallback retrieval, `assignment_context` stabilization, heuristic recommendation logic, and Streamlit RFP page integration. Real Azure-backed retrieval and final capacity-aware assignment ranking remain future integration work.

The next stage will focus on making the RFP workflow more robust and reviewable. This includes confirming Azure deployment configuration, smoke-testing the preferred embedding and retrieval path where credentials are available, preserving fallback retrieval, refining recommendation logic, improving the RFP dashboard display, and collecting screenshots and examples for the final report and demo. The project remains an applied decision-support system rather than a benchmark-driven modeling task, so future evaluation will emphasize interpretability, stakeholder review, reproducibility, smoke tests, and practical usefulness.